现场决定闭环：工地上的事自己跑，只有要拍板的才递给人

设计文档中文版 · 2026-11-05 · 对应英文原文第 3 版（已对照作者开源代码 KKKKhazix/agent-autoloop）：fieldsight-pipeline 仓库 `docs/superpowers/specs/2026-11-05-site-decision-loop-design.md`（第 1、2 版 PR #1219，流程图 #1220，第 3 版见最新 PR）· 属于架构研究，尚未开发，没有开关，也没有数据库迁移。

标签说明： 你已定 我替你定，可推翻 未定

1. 做什么，为什么

项目经理同时只能跟进三到五件事。现在 FieldSight 的所有产出（报告、漏项、待办、提醒）都推给项目经理，由他自己判断哪件重要。借鉴卡兹克文章的思路：工地上发生的事自动触发 Agent，Agent 把它有权做的做完，只把需要人判断的那一小部分做成**一键决定卡片**交给人，人的回答再作为新事件回到系统。

目标

- 工地上发生的所有事都从一个入口进来（叫"事件"），由一张规则表决定谁来处理，在一个地方记录做了什么。
- 每人每天只收到 1 到 3 张真正需要他的卡片，而不是一堆报告。
- 每张卡片都说实话：发生了什么、后果、建议、截止时间、没人回复会怎样。
- 任何不可撤回、或对外的动作，都要在有权的人同意之后才做。
- 做过的决定不再问第二遍（决定台账）。

第一版不做

- 不取代报告、Ask、行动包，它们以后都变成中台后面的"处理程序"。
- 不做通用工作流引擎，也不做让客户自己编辑规则的界面。
- 不做研发流程那张图（以后单独做）。

做成这样算失败（哪怕技术上能跑）

- 项目经理收到的提醒比以前还多。
- 卡片里引用的原话录音里没有（编造）。
- 还没批准、或者因为超时，就对外发了东西或标了"通过"。
- 任何事件、卡片、台账、记录被其他公司看到。
- 模型调用没有上限地循环。
- 安全项上"接受现状"却没有记录原因。

2. 现有代码里可以复用的（已对照 develop 核对）

现有部分	情况	在闭环里的用途
录音结束、提取	已有	事件来源；处理结果
会议决定	每次重新提取都会整体重写；但有可长期追踪的编号 `topic_decisions.stable_id`	不能直接当台账；台账用这个编号指向"关于哪件事"。prod 上编号目前只能部分延续
检查漏项	prod 已有，但**不存表**，打开报告时现算	要新写一个发出事件的地方
"查完了吗"	TEST；只在有人语音提问时运行	第一版不当后台处理程序
状态变更建议	已有：系统提议，人点接受或忽略	最接近决定卡片的现有功能
行动包	TEST 内部；以发起人的身份运行，已记录用了哪些模型	第一版保留自己的入口
兜底换模型	代码里有**三套**各自独立的逻辑	统一成"模型调用层"要改造两个客户端，推迟到第一阶段之后
token 记账	只记模型和用量，**没记公司、项目、费用**	要给每次调用带上公司信息，才能做"每个公司每天的预算"
设备在线	有心跳时间和按工作日算的"安静"提醒；冻结表还没人写入	"设备离线"要新做定时扫描
角色	公司：admin / gm / pm / worker / platform_admin；项目：pm / site_manager / worker。没有安全代表、算量员，也没有委托	第一版按现有角色用
送达渠道	邮件、设备语音；没有手机推送和 Teams	第二阶段用网页收件箱加邮件
检查"通过"状态	没有这个状态，也没有写入接口	"标为通过"移出第一版

3. 一段话说清整个闭环

> 事件进来 → 中台确认属于哪个公司、哪个项目 → 已经过时的只作事后记录 → 去重 → 匹配规则（没有规则就进"未处理池"）→ 定紧急度 → 排队 → 派给处理程序 → 处理程序读证据、调模型 → 核对：有没有证据、换一家模型复核（最多 3 轮）、只含本项目、**要不要人定** → 不用人定就内部发布并核实；要人定就写卡片 → 判断：查台账、推给有权的人、催到截止、超时走保守默认、写进台账 → 决定作为新事件回到中台。任何一步失败或超时都交回中台：第 3 次失败发故障卡给公司管理员和运维（不是项目经理），第 5 次停止重试。

4. 各部分怎么做

4.1 事件格式

每个来源都写同样一条短记录：编号、公司和项目（由中台根据来源身份确定，不信任内容里写的）、来源、类型、关于什么（录音、检查项、待办、设备、卡片）、**现场发生时间**（和收到时间分开）、一句话摘要、原始链接、连锁编号和深度。

事件只带数据，不带指令（借鉴作者）：认不出的字段直接拒收；摘要、录音原文这类外部文字永远不当作指令进提示词，只放校验过的类型、编号、链接，证据原文以"引用"的形式放进去。录音转写是外部文字，这条对我们比对作者更重要。

"待办逾期""检查到期""设备很久没上线"这类事不会自己发生，要靠**定时扫描**产生。

我替你定 第一阶段两条规则的事件来源：

- **检查漏项**：报告最终版出来时发出，只看最终提取；同一项漏项重复出现会被去重丢掉；漏项消失了，对应的卡片自动作废。
- **待办逾期**：每天扫描一次，同一条待办每天最多一件事件。

4.2 中台

1. **鉴权和归属**：已退出的公司的事件直接丢弃。
2. **过时了吗**：比如浇筑已经浇完了、设备断网六小时后才传上来，只作事后记录，不发卡片。
3. **去重**：按公司、项目、类型、对象、时间段去重。**人做出的决定永远不和原事件合并**。
4. **匹配规则**：每条规则写清处理程序、模型档次、skill、并发、通知谁、紧急度、每天上限。客户公司可以改其中几项：收卡片的人、每天上限、免打扰时段。
   没有规则的进"未处理池"，管理员页面能看到。同类事件 7 天内出现 5 次 我替你定，Agent 起草新规则，发卡片给运维批准，批准后写进规则表；否则进每周汇总。属于"必须先问人"类的，哪怕没规则也直接转人。
5. **定紧急度，排队**（紧急的不受每天上限和预算暂停限制），**派单**。
6. **收结果**：结果分四种：成功、没什么可做、失败、超时不知道做没做完。失败和超时都退避重派，间隔 5 分钟起翻倍、最长 1 小时（作者的数字）。两个门槛分开 我替你定：**第 3 次失败**发故障卡通知管理员和运维，**第 5 次失败**停止重试（作者是第 10 次停；我们每次可能调用模型，所以停得早）。等待重试期间，**同一件事新来的事件并进这一轮**，不另起一轮（借鉴作者，正好解决重跑的检查漏项重复发卡片）。成功且不用人定、或没什么可做，就结案。

4.3 处理程序和模型调用层

第一阶段的处理程序不调用模型，漏项和逾期直接根据数据生成卡片，所以不用等模型调用层。等第一个需要模型的规则上线时，再把所有模型调用统一到一层：注册表（模型和参数，改一行就换）、兜底换模型、按公司记 token 和费用、刹车。Claude 模型只走 Claude API 你已定。

4.4 核对

1. **有证据吗**：每条要给人看的内容都必须能对上录音或文档里的原话，对不上的删掉。
2. **换一家模型复核**：有问题退回修改，最多 3 轮，还不行算失败。用哪个模型复核要按这个处理程序自己的测试结果来定。
3. **只含本项目吗**：不是就**直接拦下**，告警给运维，不重试、不发布。
4. **要人定吗**：两个来源合起来判断：处理程序在结果里**自己声明**（需要人选择、需要人补信息）；中台再**用代码复核**每个下一步是否在"必须先问人"清单里，不管处理程序怎么说。任意一个说"要"就写卡片。影子阶段两个来源分开统计准确率。这道关口是我们自己加的：作者的系统没有，只是把清单写进 Agent 的提示词，而"写进提示词"不等于"会照做"。
5. **内部发布**（报告、待办、时间线）。对外发送永远不在这一步发生。
6. **发布后核实**：真的发出去了吗；重发一次，再失败算失败。

4.5 判断

1. 写卡片，带截止时间和"不回复会怎样"。
2. 查台账：定过了就照定的做；问过了且情况没变就不再发。
3. 按紧急度推给对这个项目有权的人。
4. 没送达就重试：第 0、1、3、7、15、25、35、45、55 分钟各一次（作者的时间表），再换渠道；卡片一直开着。每条发出去的消息带卡片版本号，卡片更新时改原消息，不另发新消息。
5. 截止前提醒一次，再升级（第一版升级给公司管理员）；到截止仍没人回复，按保守默认处理，台账记"超时"。
6. 回答要全部通过这几项：对得上发出去的那条消息；**只算第一次**有效回答，后来的告诉他已经定了；必须登录后回答（作者"拿到链接谁都能答"只适合一个人用）；**"有没有权替这个项目做决定"** 你已定，没有就不算数并告诉他。
7. 写进台账，变成新事件。

4.6 连锁深度

处理程序发出的新事件继承连锁编号，深度加一；到第 3 层只记录不再问，防止两个处理程序互相触发。 未定 人的回答算不算一层：作者不算（只有 Agent 发出的事件才算，上限 8 层），定时扫描从 0 开始。照这个规则，我们的 3 层只数 Agent 的跳转。

4.7 Agent 用什么身份干活 我替你定

- 每个公司、每个工地各一个系统身份，只能读这个工地的数据。
- 卡片里引用的录音按收卡人的权限再过滤一遍：工长只看到他在 App 里本来就能看到的内容。
- 真正改数据的动作（关闭待办、发邮件），只在卡片被回答后，**以回答人的身份**执行，所以 App 原有的权限保护同样管住 Agent。
- 行动包第一版保留自己的入口。

5. 数据表（草案）

- **事件表**：上面的事件格式，加状态（过时、未处理、排队、运行中、完成、没什么可做、失败、拦下）。去重编号由公司、来源、来源自己的编号组成；中台自己发出的事件加后缀，避免和来源撞号。
- **运行记录**：每次尝试的处理程序、领活编号、租约到期时间、开始结束时间、结果（完成 / 没什么可做 / 失败 / 超时未知 / 取消）、结果指纹、错误、token、费用、调了几次模型、用了哪些模型。同一结果重复上报得到同样的确认，同一次领活报来不同结果就拒收（借鉴作者，让重试和丢包都安全）。
- **卡片表**：类型（批准、选择、补信息、故障、规则）、问题、后果、选项（标推荐）、证据、紧急度、截止、超时默认、收件人、版本号、已送达的消息、状态（开着 / 已回答 → 后续动作做完后为完成 / 超时 / 作废）。卡片编号由公司、项目、规则、对象、用途组成，只在卡片开着时去重。
- **决定台账**：**只能追加**，用数据库触发器拦住修改和删除。每条写：定了什么、类型（已定 / 定了不做）、谁定的、凭什么权限、什么时候、证据、推翻了哪一条、来源（卡片 / 超时默认 / 会议）。改主意就新增一条。

6. 卡片

6.1 卡片只做三件事 你已定

批准一个必须先问人的动作；在几个选项里**选择**（附建议）；**补一个**系统推断不出来的事实。单纯的通知不发卡片。

能推断的不问 你已定：“下周三”按录音日期算出来；位置有歧义就挂到最可能的楼栋，都标成“推测，可改”。只有推断不出来、而且猜错会有后果时才问。

6.2 必须先问人的动作和超时默认 我替你定

动作	没人回复时
对公司外发送任何东西（分包商、业主）	不发
把检查标为“通过”（**第一版不做**，系统还没有这个状态）	不通过
改进度计划	不改
删除数据	不删
把录音归到某人名下	不归
安全项上接受漏项（必须填原因）	不接受

默认永远选“对外不改变任何东西、不证明任何东西”的那一边。

6.3 卡片要说实话

一句话问题、后果、证据（原话加播放按钮）、选项（标一个推荐）、截止、不回复会怎样。只说系统真能做到的：“提醒 Tom 15:00 复查并回传照片”，而不是“已安排复查”。给新西兰客户的卡片用英文。

6.4 谁收卡片

最终目标：项目经理、工长、安全代表、算量员、分包工长（对外，必须先批准）、公司管理员、运维。

我替你定 第一版先用现有角色：项目 pm = 项目经理，site_manager = 工长，公司 admin/gm = 公司管理员，platform_admin = 运维；升级从项目经理到公司管理员。安全代表、算量员两个新角色和休假委托表放到第三阶段。

6.5 例子

- 批准："给 ABC 水电发这封整改邮件吗？"（附草稿）
- 选择："浇筑前检查有 3 项没覆盖" → 提醒 Tom 复查 / 填原因接受 / 改派
- 补信息："'north wall' 是 A 栋还是 B 栋？"（只在位置表判断不了、而且是缺陷时才问）
- 冲突："周一说 C30，周三说 C35，以哪个为准？"
- 不发卡片："下周三是 11 月 12 日吗？"（能推断）

7. 紧急度 你已定

- **可能伤人：直接最高**，不参与排序，不受每天上限和预算限制，并抄送安全代表。
- 其余依次按：离截止还有多久 → 会卡住多少后续工序 → 复杂度和牵扯的人数（人越多越容易拖，要越早发）。

具体打分公式由开发定，影子阶段你在真实卡片上看权重合不合理。

8. 防止烧 token 的五道刹车 我替你定的起步值

1. 每次尝试最多调用模型 8 次。
2. 同一任务第 3 次失败通知，第 5 次停；间隔 5 分钟起翻倍、最长 1 小时（最坏 40 次调用）。
3. 连锁深度最多 3 层（只数 Agent 发出的事件，人的回答算不算待定）。
4. 每个公司每天的预算：用到 80% 提醒，用完暂停不紧急的任务。
5. 熔断：同一类错误短时间内大量出现，整条线暂停，只报一次。

任何一道碰顶就停，只发一条告警。具体数字按影子阶段的实际用量再调。

9. 项目位置表（A 栋、B 栋、单元、楼层） 未定

建议：从上传的图纸和文档里抽取，项目经理用一张卡片确认一次；录音里听到的新叫法先当候选，确认或能对上图纸后再加进去。手工填写可选，不强制。

10. 租户、隐私、保留期

- 每张表都按公司和项目区分，每次读取都按两者过滤；核对阶段的"只含本项目"是第二道保险，不是唯一一道。
- 决定台账在 HSWA 2015 事故调查和建筑纠纷里可以作为证据。**保留期和公司退出后怎么处理，你已决定放到最后（低优先级）**。我建议的默认值是项目结束后 10 年（Building Act 2004 第 393 条的最长时效），要请律师确认。在定下来之前，什么都不删。

11. 工地断网

设备和手机可能断网几个小时。事件带现场发生时间，中台的"过时了吗"会拦住过期卡片；推送没送达就换渠道；设备语音只在设备在线时用。

12. 分阶段上线

阶段	做什么	进入下一阶段的标准
第一阶段：影子	中台、规则、台账表；两条不调模型的规则（检查漏项、待办逾期）；卡片只生成**不发出**，你在审核页上打标签	你标过至少 50 张：该给人的判断准确率 ≥80%（处理程序自己声明的、中台复核出的分开算），编造证据 0，每人每天 ≤3 张。工程标准（借鉴作者）：独立的重放校验、不变量检查（没有无来由的卡片、没有回答被算两次、没有跨公司的数据）、故意改坏代码必须全部被测试抓到，每晚跑一次
第二阶段：内部	在 Test Project 上发给我们自己的人，用网页收件箱加邮件	上面的指标连续保持 2 周；超时默认和"项目经理 → 公司管理员"升级都正确触发
第三阶段：一个客户	一个试点公司的项目经理和工长	项目经理评"有用"≥70%；审计确认没有未经批准的对外动作
之后	更多规则（设备、检查到期、天气）、更多角色	每条规则单独过

prod 开关、数据库迁移、选哪个试点客户都由你定。影子审核页、卡片收件箱、未处理池这些网页功能是前端的活，前端 main 由你合，所以它们决定第一阶段的审核和第二阶段什么时候能开始。

13. 已定、我替你定的、未定

你已定 卡片三种用途；能推断的不问；"有权替项目决定"而不是"是不是项目经理"；紧急度四个维度、可能伤人直接最高；保留期延后；Claude 模型只走 Claude API。

我替你定，可推翻 收卡片的角色和第一版用现有角色；超时保守默认；未处理池 7 天 5 次的门槛；刹车数字；Agent 的系统身份方案；第一阶段两条规则的事件来源，以及第一阶段不调模型；"标为通过"移出第一版；第二阶段用网页收件箱加邮件；行动包保留自己的入口。

未定 项目位置表的来源；人的回答算不算连锁一层；每个角色优先用哪个渠道；免打扰时段。

14. 对照作者开源代码（第 3 版，2026-11-05）

文章里的系统已开源：github.com/KKKKhazix/agent-autoloop（MIT 协议，版本 ee74f43）。我们只读了代码，没有运行。它是**一个人、一台机器**用的系统（Node + SQLite），中台本身从不调用模型，只负责收事件、分派、派活、记账。

我们说对了的

必须先问人的默认清单、按来源和编号去重、长轮询领活、退避重派、第 3 次失败发通知、回答变成新事件并接回原来的对话、只能追加的日志是唯一权威记录、事件 / 轮次 / 卡片 / 送达各一张表。

被文章误导的

我们以为	实际
中台有"要不要人定"的关口	没有，由 Agent 自己判断，先问清单只是提示词里的一句话
失败 3 次就转人	第 3 次通知，第 10 次才停
连锁最多 3 层	上限 8 层，人的回答不算一层
代码审查最多 3 轮	每次提交只审一次
全新虚拟机跑完整检查	等项目自己的 CI
自动合并	先关掉 GitHub 自动合并，核对代码没变后自己合
"已定 / 不做"是系统机制	只是两份 Markdown 文件的约定
未匹配事件能起草规则	只列出最近 20 条

这一版借鉴进来的

重试时合并同一件事的新事件；"没什么可做"和"超时未知"两种状态；结果指纹防重复；中台事件编号加后缀；卡片版本号、送达重试时间表、只在开着时去重；回答要对上发出的消息、只算第一次；事件只带数据；重放校验、不变量、故意改坏测试作为第一阶段工程标准；退避的具体数字。

没借鉴的

拿到链接不登录就能回答、按渠道配置谁能回答（公司要按项目权限）；花费只展示不拦截（我们要刹车，只借它的统计口径）。

只有我们有、要自己验证的

多租户；过时事件检查；紧急度和受伤优先；中台的先问人关口；截止 → 提醒 → 升级 → 保守默认；回答权限检查；触发器保证台账只追加；五道刹车；换家复核；未匹配池起草规则；模型调用层；卡片三种用途；能推断的不问。

15. 留给开发决定的

事件用什么传（队列还是轮询表）、事件表放在哪个数据库以及旧数据怎么归档、紧急度打分公式、处理程序怎么声明"需要人"、卡片在各渠道上怎么显示。